Question 1a.

CODE:

```
# Run the code using : python ecode.py

# Mapping functions
# Converts an uppercase letter ( A to Z ) to a number ( 0 to 25)
def letter_to_number(letter):
    return ord(letter) - ord('A')


# Converts a number ( 0 to 25 ) back to its respective uppercase letter
( A to Z )
def number_to_letter(number):
    return chr(number + ord('A'))


# Encryption function


def encrypt(plaintext, key):
    # Remove spaces and convert everything to uppercase for consistency
    plaintext = plaintext.replace(" ", "").upper()
    key = key.replace(" ", "").upper()

    # Check if the key is long enough; if not, raise an error
    if len(key) < len(plaintext):
        raise ValueError("Key must be at least as long as the
plaintext.")

    # Trim the key to match the length of the plaintext
    key = key[:len(plaintext)]
    cipher = ""  # Initialize an empty string to build the ciphertext


    # Encrypt each character
    for p_char, k_char in zip(plaintext, key):
        # Convert plaintext and key characters to numbers 0 to 25
        p_num = letter_to_number(p_char)
```

```
        k_num = letter_to_number(k_char)

        # Add the numbers then mod 26
        c_num = (p_num + k_num) % 26

        # Convert the result back to a letter and append to the
ciphertext
        cipher += number_to_letter(c_num)

    return cipher  # Return the fully encrypted ciphertext

# User Input Section

# Ask user to enter the plaintext
plaintext = input("Enter plaintext (A-Z only, no punctuation): ")

# Ask user to enter the key ,must be at least as long as the plaintext
key = input("Enter key (A to Z only, must be >= length of plaintext):
")

# Try to encrypt the plaintext using the given key
try:
    cipher_text = encrypt(plaintext, key)  # Encrypt the input

    # Display the the instructions & results
    print("\n--- Encryption Output ---")
    print("Plaintext:", plaintext.upper())
    print("Key used: ", key[:len(plaintext)].upper())
    print("Ciphertext:", cipher_text)

# If there's an error (like key being too short), print the error
message
except ValueError as ve:
    print("Error:", ve)
```

OUTPUT:

```
PS C:\Users\vinay\Downloads\Comsec code> python ecode.py
Enter plaintext (A to Z only, no punctuation): VINAYA
Enter key (A to Z only, must be >= length of plaintext): THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM

--- Encryption Output ---
Plaintext: VINAYA
Key used:  THISIS
Ciphertext: OPVSGS
```

Brief Explanation:

My code converts each letter of the plaintext and key into numerical values from 0 to 25 corresponding to their position in the alphabet, then combines the numbers by adding them and modulo 26 them to produce the ciphertext. The key must be at least as long as the plaintext to ensure proper encryption. The program cleans the input by removing spaces and converting letters to uppercase, then processes each character pair from the plaintext and key to generate an encrypted message. Finally, it outputs the original plaintext, the trimmed key used, and the resulting ciphertext, while handling cases where the key is too short with an error message.

Question 1b.

The decryption of the encrypted message uses the same key and a reverse of the encryption operation process. Since encryption adds the numerical values of the plaintext and key letters modulo 26, decryption subtracts the key's numerical values from the ciphertext letters, also modulo 26. Specifically, for each character, the plaintext letter is recovered by calculating **(c_num - k_num + 26) mod 26** first. The addition of 26 ensures the result stays within the valid range and avoids negative values before applying the modulo operation. This process reverses the encryption and gets the original plaintext. Thus, the same key that was used to encrypt the message is required to correctly decrypt it, demonstrating the principle of symmetric key encryption.

CODE :

```python
# run code like this : python dcode.py

#  Mapping functions
# Convert an uppercase letter (A to Z) to a number (0 to 25)
def letter_to_number(letter):
    return ord(letter) - ord('A')


# Convert a number (0 to 25) back to an uppercase letter (A to Z)
```

```
def number_to_letter(number):
    return chr(number + ord('A'))


#  Decryption function


def decrypt(ciphertext, key):
    # Remove spaces and convert both the ciphertext and the key to
uppercase
    ciphertext = ciphertext.replace(" ", "").upper()
    key = key.replace(" ", "").upper()


    # Ensure the key is at least as long as the ciphertext else error
    if len(key) < len(ciphertext):
        raise ValueError("Key must be at least as long as the
ciphertext.")


    # Trim the key to match the length of the ciphertext exactly
    key = key[:len(ciphertext)]
    plaintext = ""  # Initialize an empty string for the decrypted
message


    # Decrypt each letter
    for c_char, k_char in zip(ciphertext, key):
        c_num = letter_to_number(c_char)  # Convert ciphertext letter
to number
        k_num = letter_to_number(k_char)  # Convert key letter to
number


        # Subtract key number from ciphertext number, add 26 to avoid
negative then mod 26 it
        p_num = (c_num - k_num + 26) % 26


        # Convert the result number back to a letter and append to
plaintext
        plaintext += number_to_letter(p_num)
```

```
    # Return the decrypted plaintext, cleaned ciphertext, and trimmed
key
    return plaintext, ciphertext, key


# Main Program


try:
    # Take user input for ciphertext and key, allowing spaces for
readability
    ciphertext_input = input("Enter ciphertext (A to Z only, spaces
allowed): ")
    key_input = input("Enter key (must be >= length of ciphertext,
spaces allowed): ")


    # Perform decryption using the provided inputs
    plaintext_result, clean_ciphertext, trimmed_key =
decrypt(ciphertext_input, key_input)


    # Display original inputs and the decryption result clearly
    print("\n--- Decryption Output ---")
    print("Original Ciphertext : ", ciphertext_input)
    print("Cleaned Ciphertext  : ", clean_ciphertext)
    print("Key used            : ", trimmed_key)
    print("Decrypted Plaintext : ", plaintext_result)


except ValueError as ve:
    # Handle errors such as key length issues gracefully by printing
the error message
    print("Error:", ve)
```

OUTPUT:

```
PS C:\Users\vinay\Downloads\Comsec code> python dcode.py
Enter ciphertext (A to Z only, spaces allowed): OPVSGS
Enter key (must be >= length of ciphertext, spaces allowed): THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM

--- Decryption Output ---
Original Ciphertext :  OPVSGS
Cleaned Ciphertext  :  OPVSGS
Key used            :  THISIS
Decrypted Plaintext :  VINAYA
```

Brief Explanation:

converts letters to numbers from 0 to 25, then for each character in the ciphertext, it subtracts the corresponding key character's number, wraps the result within the alphabet range, and converts it back to a letter to show the original plaintext. The key must be at least as long as the ciphertext to ensure accurate decryption. The program also cleans inputs by removing spaces and converting to uppercase. It then outputs the original ciphertext, the cleaned ciphertext and key used, and the decrypted plaintext, while handling errors like a key that's too short.

Question 1c

1. Vulnerability to Known Plaintext Attacks :

If an attacker gets part of the plaintext and the matching ciphertext, they can use this to figure out the caesar cipher's shift key. This kind of attack is called a known plaintext attack.**[1]**

Example: Let's say if a Caesar cipher message always starts its messages with the word "CONFIDENTIAL" and If an attacker intercepts an encrypted message that begins with "FRQILGHQWLDO", and suspects that it corresponds to "CONFIDENTIAL", the attacker can compare each letter in the suspected plaintext and ciphertext to determine the shift. From here, the attacker will find that each letter has been shifted by 3 which means that the key is 3.

Reason: Since Caesar cipher uses the same shift for every letter and doesn't involve complex transformations, a small amount of known plaintext is enough to figure out the full encryption. Known plaintext attacks work best when the encryption method always uses the same key and doesn't add any randomness to how the message is encrypted. In the Caesar cipher, the key stays the same for the whole message, and every letter is shifted in the same way. That's why the Caesar cipher is especially vulnerable to this kind of attack. **[1]**

2. Weak or Reused Key:

Reusing the same key for multiple messages makes it easier for hackers to spot patterns and break the encryption, so it's important that each key is used only once. Weak keys like ones that are too short or not truly random are also risky because they're easier to guess.

That’s why strong keys should always be generated using reliable tools that create random values based on real, unpredictable sources. **[2]**

Example: We will take Caesar cipher as an example again. Let’s say two encrypted messages both use a shift of 5. If an attacker cracks one message using frequency analysis or brute force, the same key can be used to decrypt any future message that also uses shift 5. For instance, once the attacker knows that “MJQQT” is “HELLO” which is a shift of 5, they can use that shift on all messages encrypted the same way.

Reason: Because the caesar cipher only has 25 possible keys, it is extremely vulnerable to brute force and pattern analysis. When a weak or reused key is involved, the attacker doesn’t even need to break each message individually. Reusing weak keys makes encrypted data more predictable and easier to break. This lowers the level of protection provided by the encryption method severely .**[2]**

Question 1d

1. Key Length:

Key length refers to the number of bits in a cryptographic key and directly affects how secure an encryption system is. Longer keys significantly increase the number of possible combinations an attacker would need to guess, making brute force attacks practically impossible. For example, while a 128 bit key already offers strong protection, a 256 bit key increases the difficulty of decryption a lot . Strong key lengths are important and needed for protecting sensitive data, especially in sectors like banking where confidentiality is very critical. Short or outdated key lengths can expose systems to severe vulnerabilities, so it's important to regularly assess and update key lengths based on modern encryption standards **[3]**

2. Randomness and Entropy: A secure encryption key must be generated with high entropy, meaning it should be as unpredictable and random as possible. Entropy measures the level of uncertainty in a key, and the higher it is, the harder it becomes for an attacker to guess or duplicate the key. If a key follows a pattern or is created using weak or predictable methods like common phrases, it becomes vulnerable to brute force or pattern based attack. To make sure encryption keys are safe and impossible to guess, it's important to use strong random generators especially the ones built for security so no one can figure out how the key was created or try to recreate it. Low entropy significantly weakens encryption strength, while high entropy ensures that keys remain resistant to statistical guessing and predictive models.**[4]**

3. Uniqueness and Non-Reuse:

To ensure the cryptographic key is secure, it must be used only once per session or message. Reusing the same key or values like nonces (one time use number) can create patterns that attackers can exploit. For an example, if multiple messages are encrypted with the same key or nonce, an attacker may be able to analyze the similarities in the ciphertexts

to find out the original data. Nonces are designed to ensure uniqueness and prevent replay attacks. By assigning a new and unpredictable value to each session, the system will make it very hard for attackers to reuse captured data. **[5]**

Together, these factors significantly enhance the security of symmetric encryption algorithms by making it extremely difficult for attackers to recover the key or plaintext.

Question 2a

Data Encryption Standard is a method used to encrypt data in fixed-size chunks, and each chunk must be exactly 8 bytes (64 bits) long. But in real life, most messages are not that exact as they come in different lengths.**[6]** That's where padding comes in

Purpose of Padding:

Padding ensures that the input data is properly aligned to fit DES's block size requirements. If the final block of plaintext is shorter than 64 bits, extra data known as padding must be added so that it reaches the required length. Without padding, DES would be unable to encrypt or decrypt the final block correctly. **[7]**

Why Padding Matters:

- ➢ It ensures the final block is full, so the algorithm can process it. **[7][8]**
- ➢ Padding helps prevent certain attacks by hiding the actual length of the message and avoiding repetition patterns. **[8][10]**
- ➢ Padding helps keep the data in the proper format so that everything works properly when encrypting and decrypting, preventing information loss. **[9]**

Example: For instance we have a plain text plaintext: "DOCTOR" which is 6 characters (48 bits, 6 bytes). DES needs 64 bits which is 8 bytes, so we are 2 bytes short hence we add 0x02. After decryption, the receiver sees the last byte is 0x02, so they remove the final two bytes to recover the original plaintext "DOCTOR".

Conclusion: Padding is essential in DES to ensure that all plaintext can be securely encrypted in complete 64-bit blocks. It maintains both functionality and security in block cipher encryption.

Question 2b

Step 1:

Replace "YYY" with the first 3 letters of my name

My name is vinaya so the first 3 letters would be: **VIN**

VIN and the Plaintext becomes: **VINComputerSecurity**

Step 2:

DES Block Size

Note that DES works on 64 bit blocks which is equal to 8 bytes per block.

Each character is 1 byte (8 bits) .

Step 3:

Characters in **VIN** = **3 characters**

Characters in **ComputerSecurity** = **16 characters**

Total characters = 3 + 16 = 19

19 characters = 19 bytes

Step 4:

Padding required

Block 1 : 8 bytes

Block 2 : 8 bytes

19 bytes - 8 bytes - 8 bytes = 3 bytes (remaining)

Block 3 : 3 bytes
Since block 3 is short of 5 bytes, it requires padding hence we will add 0x05 (0000000000).

Step 5:

Convert characters to Hexadecimal :

[13] Breakdown **VINComputerSecurity** into its respective hex values:

V = 56, **I** = 49, **N** = 4E, **C** = 43, **o** = 6F, **m** = 6D, **p** = 70, **u** = 75, **t** = 74, **e** = 65, **r** = 72, **S** = 53, **e** =65, **c** = 63, **u** = 75, **r** = 72, **i** = 69 , **t** = 74, **y** = 79

Add the padding : 0x05

Step 6:

Group into blocks of 8 bytes :

Block 1: 56 49 4E 43 6F 6D 70 75

Block 2: 74 65 72 53 65 63 75 72

Block 3: 69 74 79 00 00 00 00 00

Step 7:

Final Hexadecimal String

56494E436F6D707574657253656375726974790000000000

Question 2c

Step 1: Convert the plaintext to ASCII number [13]

We cannot encrypt the text directly in RSA so we convert each character into ASCII Number.

C = 67, **o** = 111, **m** = 109, **p** = 112, **u** = 117, **t** = 116, **e** = 101, **r** = 114,**S** = 83, **e** = 101, **c** = 99, **u** = 117, **r** = 114, **i** = 105, **t** = 116, **y** = 121

Step 2: Choosing Two Prime Numbers [11]

Usually larger prime numbers are used but for this demonstration i will smaller prime numbers :

p = 47
q = 59

Step 3 : Compute N [11]

Formula : N= p x q
N= 47 x 59 = **2773**

Step 4: Compute Euler's Totient Function [11]

Purpose : to see how many positive integers are there which are less than or equal to n which does not share any common factors with it other than 1.

Formula : ϕ**(n) = (p-1)(q-1)**
ϕ**(n)** = 46 x 58 = **2668**

Step 5: Choose public Key

C=M^e mod N

Public key : (e,n) for encryption[11]

This is something to take note**[11]** :

1) e should be between 1 and ϕ(n)
   1<e<ϕ(n)

Lets choose the public key to be 17 **[12]**

2) gcd(e,ϕ(n)) = 1

gcd(17,2668) = 1, hence e = 17 is valid to use

Step 6 : Apply padding and break blocks

In step 1 we converted the plain text to ASCII

C = 67, **o** = 111, **m** = 109, **p** = 112, **u** = 117, **t** = 116, **e** = 101, **r** = 114,**S** = 83, **e** = 101, **c** = 99, **u** = 117, **r** = 114, **i** = 105, **t** = 116, **y** = 121

Now since n = 2773 and its 4 digits , we will break the message into 3 digit blocks so that each number is less than n.

067, 111, 109, 112, 117, 116, 101, 114, 083, 101, 099, 117, 114, 105, 116, 121

Step 7: Encrypt each 3 digit block

C = M^e mod(N)

C = (67^17) mod 2773 = 2248 (C)
C = (111^17) mod 2773 = 131 (o)
C = (109^17) mod 2773 = 170 (m)
C = (112^17) mod 2773 = 1084 (p)
C = (117^17) mod 2773 = 1769 (u)
C = (116^17) mod 2773 = 2504 (t)
C = (101^17) mod 2773 = 758 (e)
C = (114^17) mod 2773 = 1684 (r)
C = (83^17) mod 2773 = 1471 (S)
C = (101^17) mod 2773 = 758 (e)
C = (99^17) mod 2773 = 2576 (c)
C = (117^17) mod 2773 = 1769 (u)
C = (114^17) mod 2773 = 1684 (r)
C = (105^17) mod 2773 = 1020 (i)
C = (116^17) mod 2773 = 2504 (t)
C = (121^17) mod 2773 = 196 (y)

Final Result:

2248 131 170 1084 1769 2504 758 1684 1471 758 2576 1769 1684 1020 2504 196

Question 2d

A digital signature is a cryptographic method used to verify both the authenticity and integrity of a digital message or a document. It serves as a secure, electronic version of a handwritten signature, ensuring that the content truly comes from the claimed sender and has not been tampered with during transmission. Digital signatures are also commonly used in secure communications and software authentication. **[14]**

RSA, which is an asymmetric encryption algorithm, is good for generating digital signatures. When using RSA for this, the sender first applies a cryptographic hash function to the original message to generate a fixed length representation of the message known as a message digest. This digest is then encrypted using the sender's private key to produce the digital signature. Upon receiving the message and the signature, the receiving person can decrypt the signature using the sender's public key and compare the result with their own hash of the received message. If both match, it confirms that the message was indeed sent by the claimed individual and has not been altered. This process provides strong assurance of authenticity and integrity of the message.**[14]**

Hence RSA is widely used for digital signatures as it provides a secure and reliable way to sign and verify messages using encryption and decryption techniques designed for this purpose.

However, Data Encryption Standard cannot be used to generate digital signatures as it is a symmetric encryption algorithm, which means it uses the same key for both encryption and decryption.**[15]** As both parties must share the same key, it is difficult to uniquely identify or verify the sender of a message anyone with access to the key could have created. Additionally, DES does not involve public and private key pairs, which are essential for the secure signing and verification process that digital signatures require. .

In conclusion, RSA can be effectively used to create digital signatures due to its asymmetric nature and support for secure verification, while DES, being a symmetric algorithm, lacks the necessary key structure and is therefore unsuitable for this purpose.

References

[1] J. Singh, “Known-plaintext attacks, explained,” Cointelegraph, Aug. 24, 2023. https://cointelegraph.com/explained/known-plaintext-attacks-explained (accessed Jun. 28, 2025)

[2] Fortinet, “What Is Cryptography? Definition, Importance, Types,” Fortinet, 2023. https://www.fortinet.com/resources/cyberglossary/what-is-cryptography (accessed Jun. 28, 2025)

[3] L. E. Team, “Key Length,” Larksuite.com, May 30, 2024. https://www.larksuite.com/en_us/topics/cybersecurity-glossary/key-length(accessed Jun. 28, 2025)
[4] S. Lee, “Understanding Entropy in Cryptography,” Numberanalytics.com, 2025. https://www.numberanalytics.com/blog/ultimate-guide-to-entropy-in-cryptography(accessed Jun. 28, 2025)
[5] Anyanwu Chioma, “Cryptographic Nonce: What It Is and Its Purpose - UEEx Technology,” UEEx Technology, Oct. 30, 2024. https://blog.ueex.com/en-us/cryptographic-nonce/#elementor-toc__heading-anchor-1(accessed Jun. 28, 2025)

[6] GeeksforGeeks, “Data Encryption Standard (DES) | Set 1,” GeeksforGeeks, Aug. 17, 2018. https://www.geeksforgeeks.org/computer-networks/data-encryption-standard-des-set-1/ (accessed Jun. 28, 2025)

[7] D. Ireland, “Using padding in encryption,” Di-mgt.com.au, 2016. https://www.di-mgt.com.au/cryptopad.html (accessed Jun. 28, 2025)

[8] “What is Padding? Enhancing Cybersecurity with Data Camouflage,” Reasonlabs.com, 2023. http://cyberpedia.reasonlabs.com/EN/padding.html (accessed Jun. 28, 2025)

[9] “Padding Schemes,” HeyCoach Blog, Jan. 09, 2025. https://blog.heycoach.in/padding-schemes/(accessed Jun. 28, 2025)

[10] “IBM DataPower Gateway,” Ibm.com, Mar. 29, 2024. https://www.ibm.com/docs/en/datapower-gateway/10.5.x?topic=wizards-protection-against-padding-oracle-attacks(accessed Jun. 28, 2025)

[11] GeeksforGeeks, “RSA Algorithm in Cryptography,” GeeksforGeeks, Apr. 22, 2017. https://www.geeksforgeeks.org/computer-networks/rsa-algorithm-cryptography/ (accessed Jun. 28, 2025)

[12] “Linux on IBM Systems,” Ibm.com, Apr. 2025. https://www.ibm.com/docs/en/linux-on-systems?topic=formats-rsa-public-key-token(accessed Jun. 28, 2025)

[13] “ASCII Character Chart with Decimal, Binary and Hexadecimal Conversions,” www.eso.org. https://www.eso.org/~ndelmott/ascii.html (accessed Jun. 28, 2025)

[14] GeeksforGeeks, “RSA and Digital Signatures,” GeeksforGeeks, Dec. 30, 2020. https://www.geeksforgeeks.org/computer-networks/rsa-and-digital-signatures/ (accessed Jun. 28, 2025)

[15] “What is Data Encryption | From DES to Modern Algorithms | Imperva,” Learning Center, 2019. https://www.imperva.com/learn/data-security/data-encryption/ (accessed Jun. 28, 2025)